

实验 4 - 循环神经网络

数据科学与大数据技术专业

第 9 周

1 实验目标

- 理解循环神经网络（RNN）的基本结构与原理.
- 掌握 RNN 在序列建模中的应用方法.
- 能够使用 NumPy 实现基本的 RNN 前向与反向传播.
- 了解 LSTM 的主要组成部分及其改进点.
- 能够利用 RNN 进行字符级文本生成任务.

2 实验环境

- 操作系统: Windows、Linux 或 macOS 均可
- Python 3.7 及以上版本
- 主要依赖库: `numpy`、`matplotlib` (用于可视化)、Jupyter Notebook (推荐)
- 提供的数据集: `dinos.txt` (恐龙名字数据集)
- 辅助文件: `rnn_utils.py` (包含 RNN 相关工具函数)

3 实验步骤

3.1 环境准备

1. 安装 Python 及相关依赖库:

```
pip install numpy matplotlib
```

2. 建议使用 Jupyter Notebook 进行实验, 便于代码调试与结果展示.

3.2 理解 RNN 基本结构

- 阅读实验讲义, 理解 RNN 的输入、隐藏状态、输出的计算流程.
- 理解 RNN 的参数 (权重矩阵、偏置项) 及其在前向传播中的作用.

RNN 基本结构定义

循环神经网络 (Recurrent Neural Network, RNN) 是一类用于处理序列数据的神经网络。RNN 通过在序列的每个时间步共享参数，并引入隐藏状态 (hidden state)，能够捕捉序列中的时序依赖关系。

RNN 的基本单元结构如下：

- 输入向量 $x^{(t)}$ ：表示在时间步 t 的输入。
- 隐藏状态 $h^{(t)}$ ：表示网络在时间步 t 的记忆。
- 输出向量 $y^{(t)}$ ：表示在时间步 t 的输出。

RNN 的参数包括：

- W_{xh} ：输入到隐藏层的权重矩阵。
- W_{hh} ：隐藏层到隐藏层的权重矩阵（自连接）。
- b_h ：隐藏层偏置。
- W_{hy} ：隐藏层到输出层的权重矩阵。
- b_y ：输出层偏置。

RNN 前向传播公式

RNN 在每个时间步的前向传播计算如下：

$$h^{(t)} = \tanh(W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_h) \quad (3.1)$$

$$y^{(t)} = \text{softmax}(W_{hy}h^{(t)} + b_y) \quad (3.2)$$

其中 $\tanh$ 为双曲正切激活函数， softmax 用于多分类输出。

3.3 实现 RNN 前向传播

1. 使用 NumPy 实现单步 RNN 单元的前向传播函数。
2. 扩展为整个序列的前向传播，输出所有时间步的隐藏状态和输出。
3. 可参考 `rnn_forward` 函数的实现。

单步 RNN 前向传播定义

给定输入 $x^{(t)}$ 和前一时刻隐藏状态 $h^{(t-1)}$ ，单步 RNN 前向传播公式为：

$$h^{(t)} = \tanh(W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_h) \quad (3.3)$$

$$y^{(t)} = \text{softmax}(W_{hy}h^{(t)} + b_y) \quad (3.4)$$

代码实现要点：

- 使用 NumPy 实现上述公式。

- 注意输入、权重矩阵和偏置的维度匹配。
- $\tanh$ 和 softmax 可分别用 NumPy 的 `np.tanh` 和自定义 `softmax` 函数实现。

序列 RNN 前向传播

对于长度为 T 的输入序列 $\{x^{(1)}, x^{(2)}, \dots, x^{(T)}\}$, RNN 前向传播递归计算所有时间步的隐藏状态和输出:

$$h^{(t)} = \tanh(W_{xh}x^{(t)} + W_{hh}h^{(t-1)} + b_h) \quad (3.5)$$

$$y^{(t)} = \text{softmax}(W_{hy}h^{(t)} + b_y) \quad (3.6)$$

3.4 实现 RNN 反向传播

1. 理解反向传播的计算流程, 掌握梯度的传播方式.
2. 使用 NumPy 实现 RNN 的反向传播函数, 计算各参数的梯度.
3. 可参考 `rnn_backward` 函数的实现.

RNN 反向传播定义

RNN 反向传播 (Backpropagation Through Time, BPTT) 用于计算损失函数对各参数的梯度。损失函数常用交叉熵损失:

$$L = - \sum_{t=1}^T \sum_{k=1}^K y_k^{(t)} \log \hat{y}_k^{(t)} \quad (3.7)$$

其中 $y_k^{(t)}$ 为真实标签, $\hat{y}_k^{(t)}$ 为模型预测。

反向传播递推公式:

$$dL/dW_{hy} = \sum_{t=1}^T (\hat{y}^{(t)} - y^{(t)})(h^{(t)})^T \quad (3.8)$$

$$dL/db_y = \sum_{t=1}^T (\hat{y}^{(t)} - y^{(t)}) \quad (3.9)$$

$$dL/dh^{(t)} = W_{hy}^T (\hat{y}^{(t)} - y^{(t)}) + W_{hh}^T \delta^{(t+1)} \odot (1 - (h^{(t)})^2) \quad (3.10)$$

$$dL/dW_{xh} = \sum_{t=1}^T \delta^{(t)} (x^{(t)})^T \quad (3.11)$$

$$dL/dW_{hh} = \sum_{t=1}^T \delta^{(t)} (h^{(t-1)})^T \quad (3.12)$$

$$dL/db_h = \sum_{t=1}^T \delta^{(t)} \quad (3.13)$$

其中 $\delta^{(t)} = dL/dh^{(t)} \odot (1 - (h^{(t)})^2)$, $\odot$ 为 Hadamard 积。

3.5 构建字符级文本生成模型

1. 加载 `dinos.txt` 数据集, 统计字符表, 建立字符到索引的映射.
2. 设计并实现基于 RNN 的字符级语言模型.
3. 训练模型, 生成新的恐龙名字.
4. 实现梯度裁剪 (gradient clipping) 以防止梯度爆炸.

字符级语言模型定义

字符级 RNN 以字符为基本单元, 输入为字符的独热编码 (one-hot), 输出为下一个字符的概率分布。

字符到索引的映射:

- 统计数据集中所有出现的字符, 建立字符表 $\mathcal{C}$ 。
- 建立映射 $\text{char2idx} : \mathcal{C} \rightarrow \{0, 1, \dots, |\mathcal{C}| - 1\}$ 。
- 输入 $x^{(t)}$ 为 $|\mathcal{C}|$ 维独热向量。

损失函数:

$$L = - \sum_{t=1}^T \sum_{k=1}^{|\mathcal{C}|} y_k^{(t)} \log \hat{y}_k^{(t)} \quad (3.14)$$

梯度裁剪 (Gradient Clipping): 为防止梯度爆炸, 对所有参数的梯度进行裁剪:

$$\text{if } \|g\|_2 > \tau, \quad g \leftarrow \tau \cdot \frac{g}{\|g\|_2} \quad (3.15)$$

其中 g 为梯度, τ 为裁剪阈值。

3.6 结果分析与可视化

- 观察模型生成的恐龙名字, 分析其合理性与多样性.
- 可选: 绘制训练损失曲线, 分析模型收敛情况.

损失曲线绘制

训练过程中可记录每轮损失 L , 用 `matplotlib` 绘制损失随迭代次数的变化曲线, 分析模型收敛性。

4 实验作业

1. 补全代码.
2. (可选) 复现某个 RNN 的开源项目, 并附上本地运行结果截图.